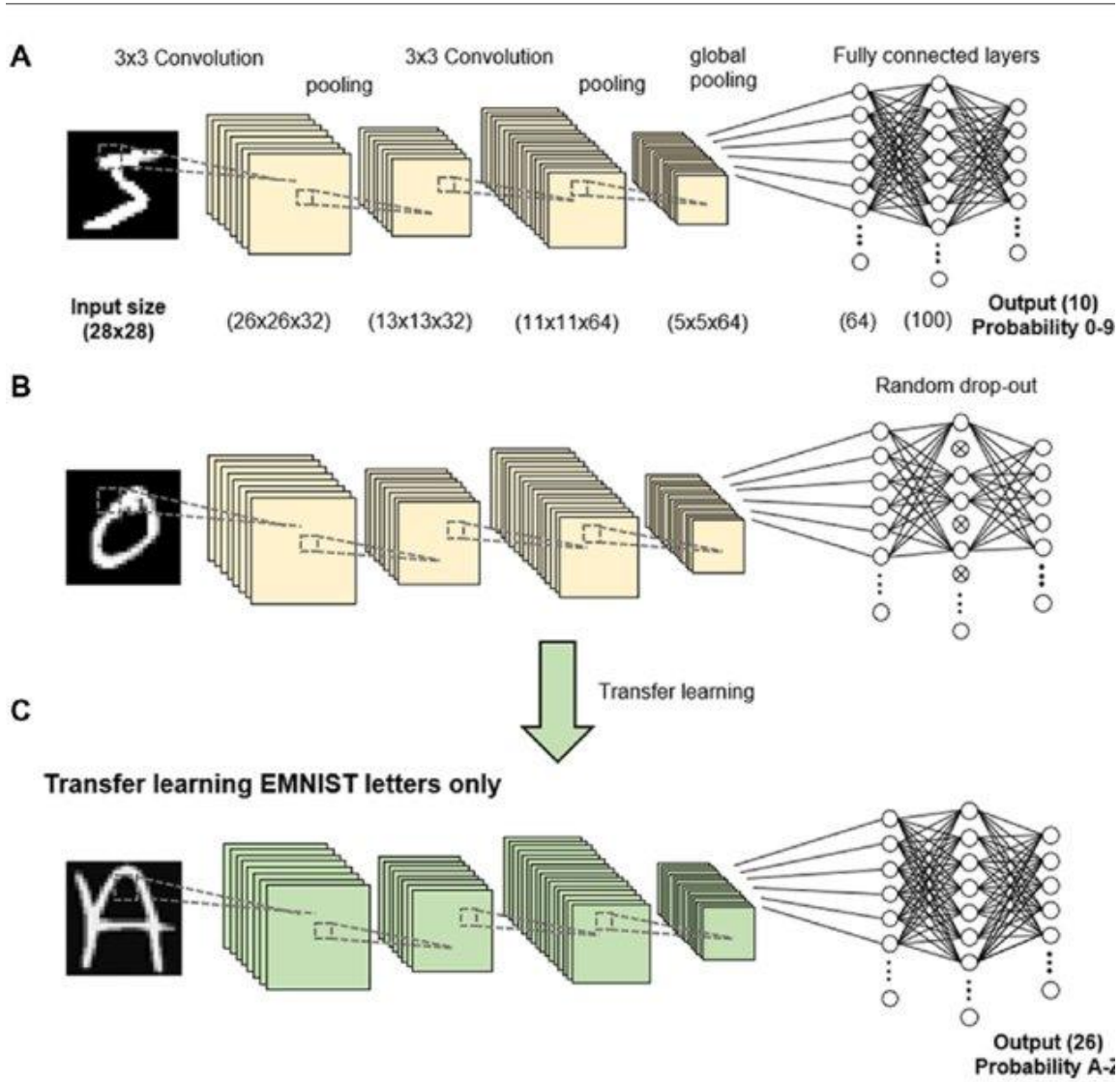
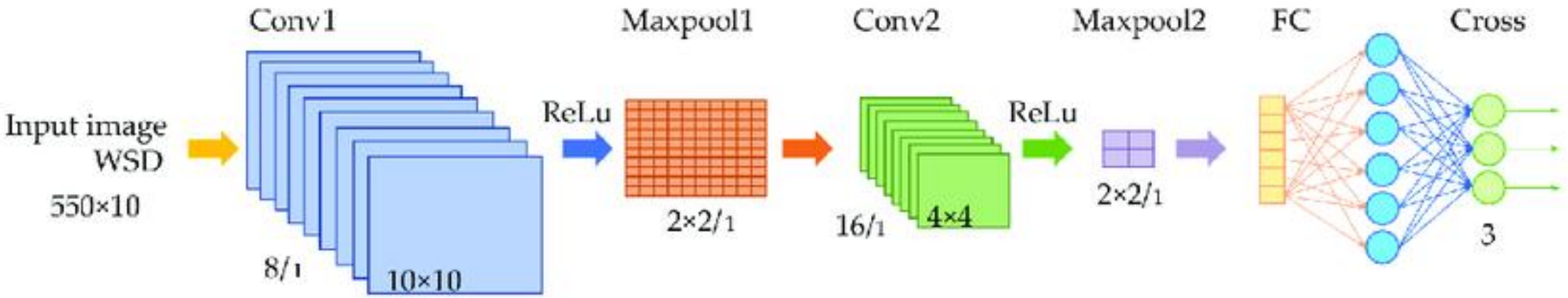
What is a Convolutional Neural Network (CNN)?

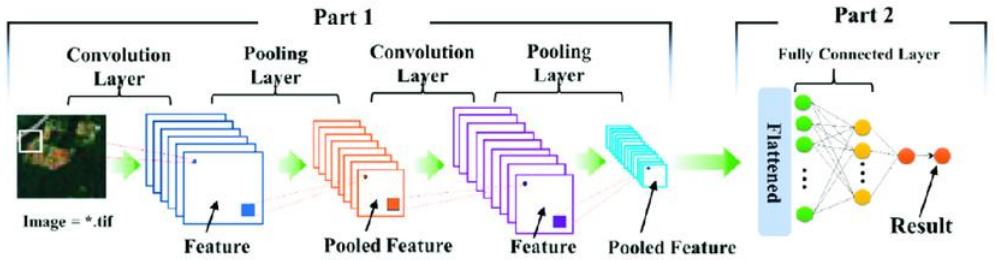
Professor Jamshid Dehmeshki

Convolutional Neural Network (CNN)

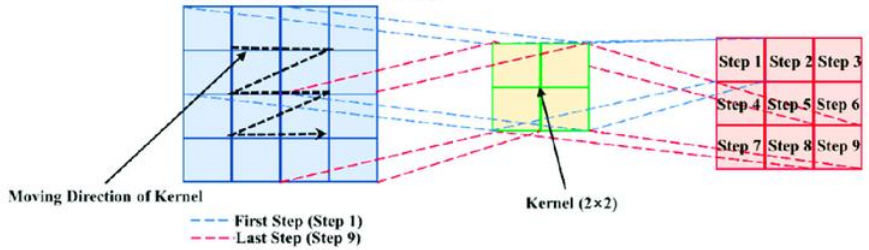
- A Convolutional Neural Network (CNN or ConvNet) is a type of deep learning model specifically designed to process data that has a grid-like topology, such as images.
- CNNs are particularly effective for tasks involving image and video recognition, classification, segmentation, and even natural language processing.



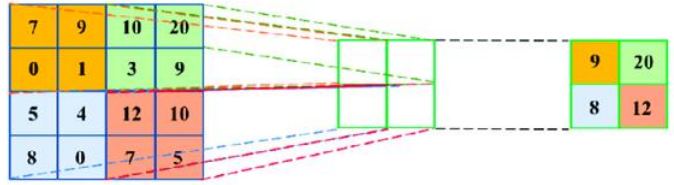

Input image
WSD
550×10
Conv1
8/1
10×10
ReLu
Maxpool1
2×2/1
Conv2
16/1
4×4
ReLu
Maxpool2
2×2/1
FC
Cross
3



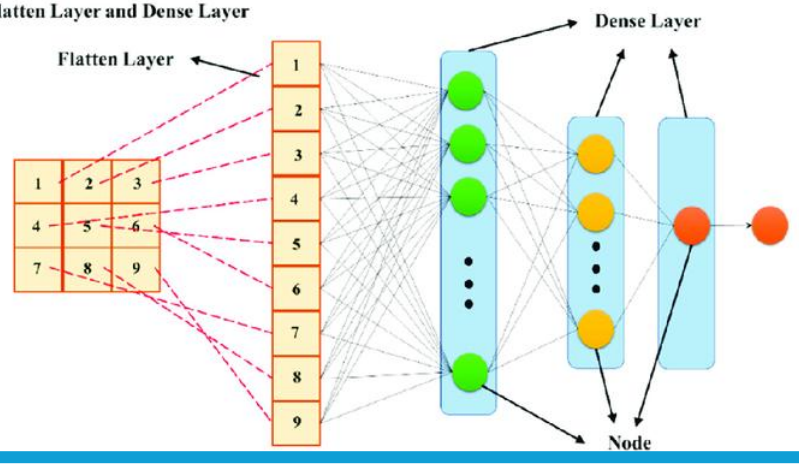
(a) Convolution layer - Feature Extraction from Image
(Stride =1, Kernel size = 2×2)



(b) Pooling Layer - Down Sampling of Feature
(Max Pooling, Stride = 2)



(c) Flatten Layer and Dense Layer



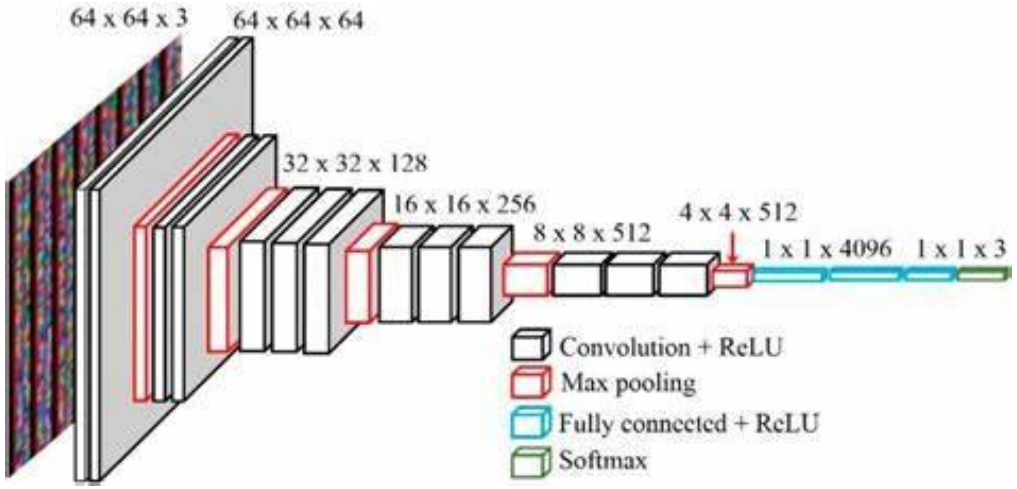
Why CNNs Matter

- Unlike traditional machine learning methods like SVMs or decision trees, CNNs can automatically identify and extract features on a large scale, eliminating the need for manual feature engineering and thus improving efficiency.
- The convolutional layers in CNNs give them the ability to detect patterns and features in data, regardless of shifts in position, orientation, scale, or translation, making them highly adaptable.

Image features are
captured from the image
1-st feature f_1^{CNN}
2-nd feature f_2^{CNN}
n-th feature f_n^{CNN}
Distorted
image patch
Classifier in CNN
Full connected
neural network
Predicted image quality, q_{CNN}
Convolution and pooling layers

Why CNNs Matter

- Several pre-trained CNN models, such as VGG-16, ResNet50, Inceptionv3, and EfficientNet, have proven to deliver exceptional performance.
- These models can be fine-tuned with relatively small amounts of data to address new tasks.



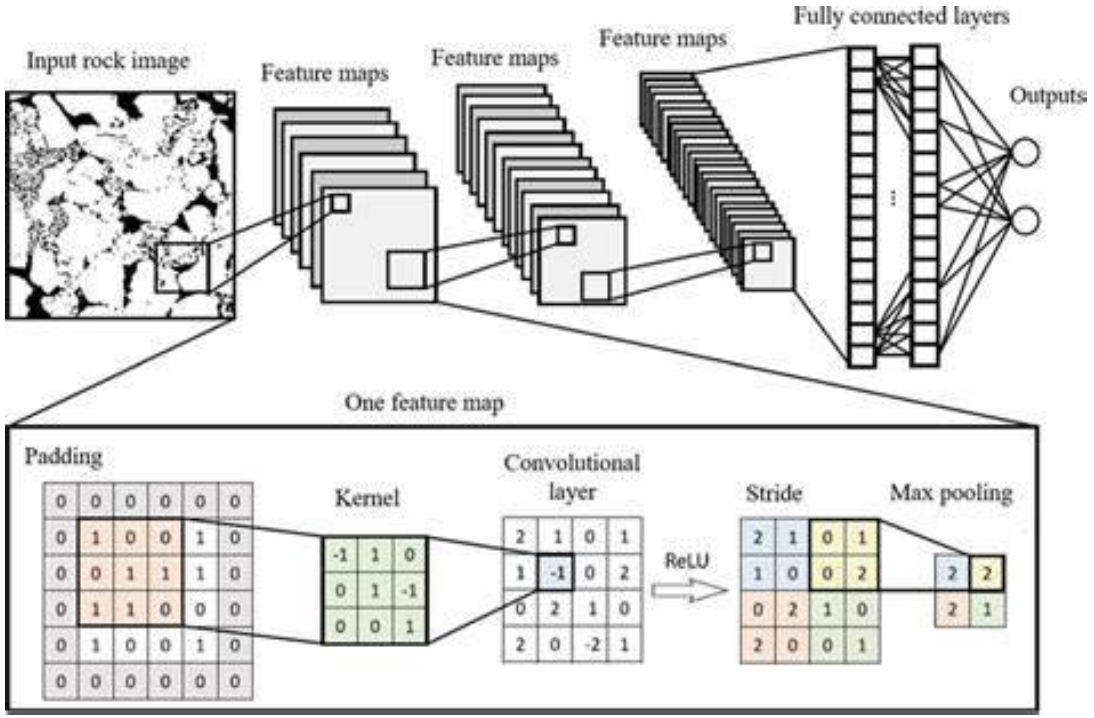
Why CNNs Are Essential:

- **Fundamental Building Block**:
  - CNNs are designed specifically to process grid-like data such as images.
  - They use convolutional layers to automatically and adaptively learn spatial pyramids of features from input images.
  - This makes them particularly well-suited for tasks involving visual data.

Why CNNs Are Essential:

- **Feature Extraction**: The core idea of CNNs is to use convolutional layers to extract features from images, which are then used by fully connected layers to make predictions.
- These features might include edges, textures, shapes, and higher-level representations like parts of objects.

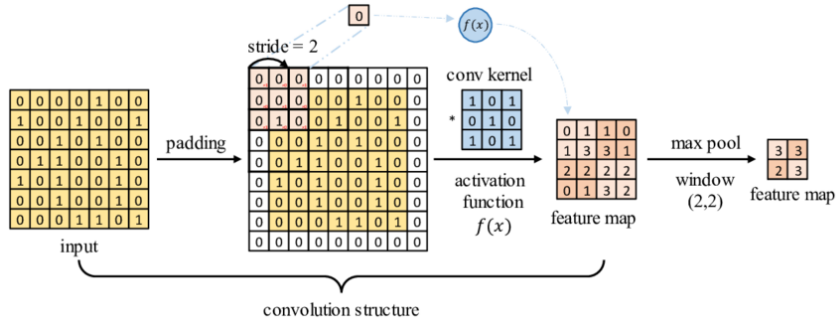
- **Versatility**: CNNs can be adapted for a variety of tasks beyond classification, such as object detection, segmentation, and even generating new images.



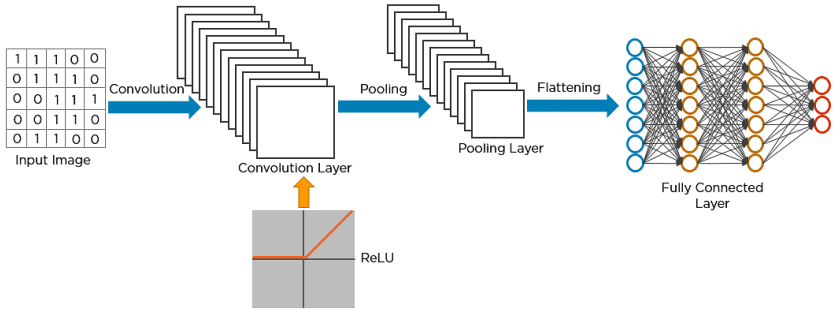
How CNNs Work:

- **Feature Extraction**: Early convolutional layers focus on learning low-level features such as edges or textures. As you go deeper into the network, the layers learn more complex features that are combinations of the low-level features.
- **Classification**: After feature extraction, the fully connected layers interpret the high-level features and classify the inpu

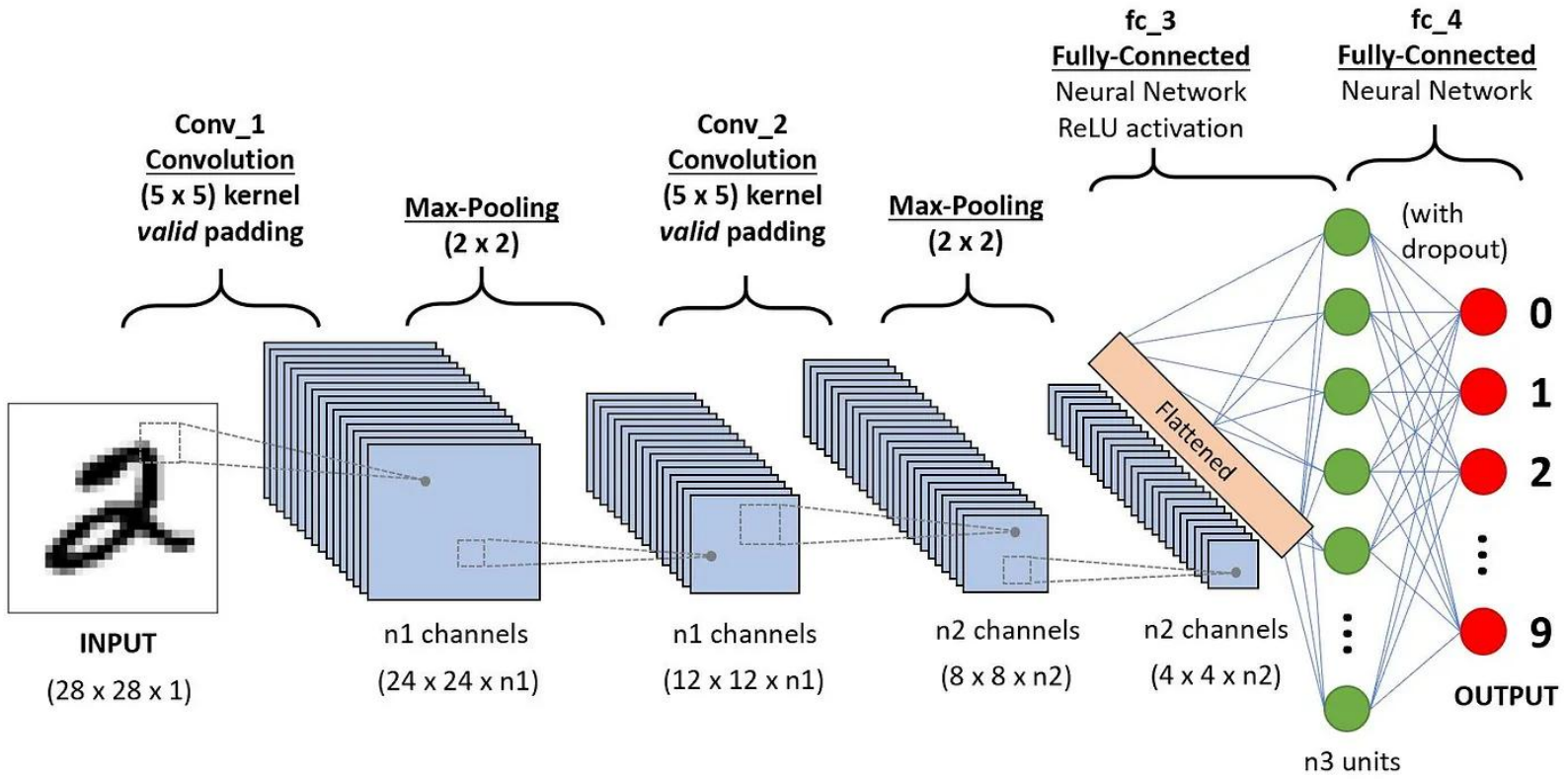
Key Components of a CNN



- A convolutional neural network consists of five primary components.
- They allow CNNs to replicate the way the human brain recognises patterns and features in images:
    1. Convolutional layers
    2. Activation functions
    3. Pooling layers
    4. Fully connected layers
    5. Dropout



- But how do these parts enable CNNs to learn?



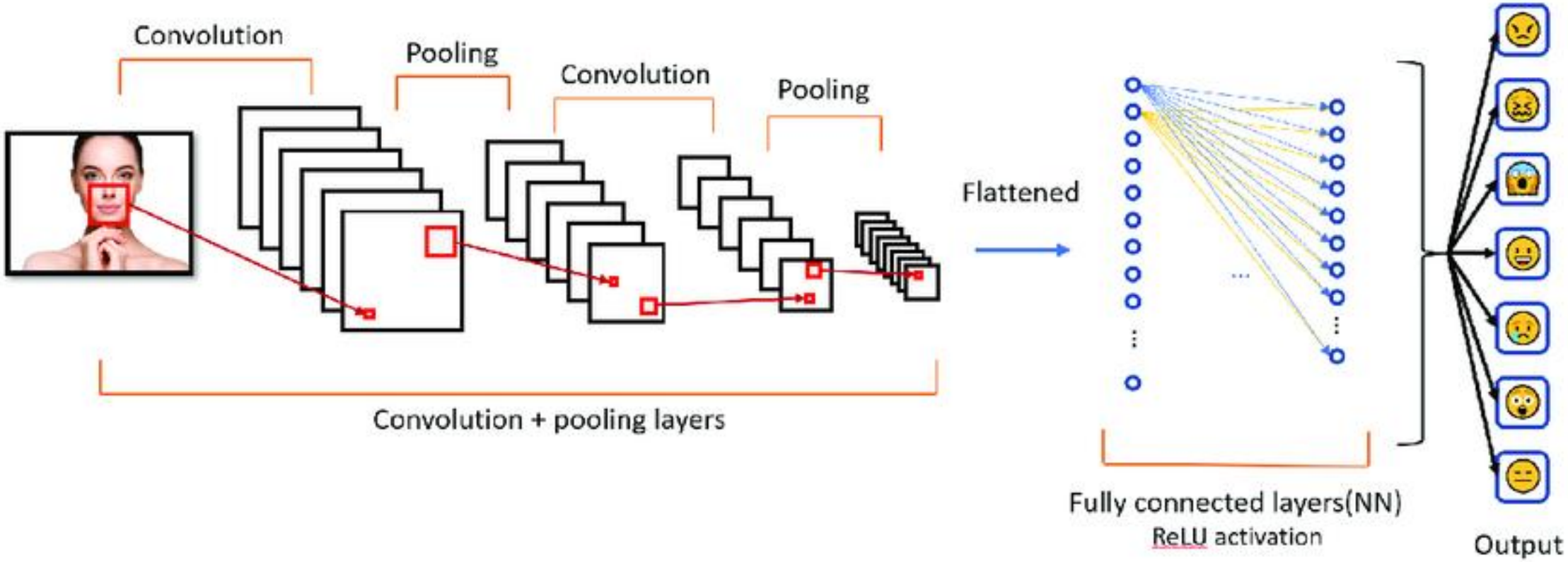
*Architecture of the CNNs applied to digit recognition (**source**)*



Convolutional layers

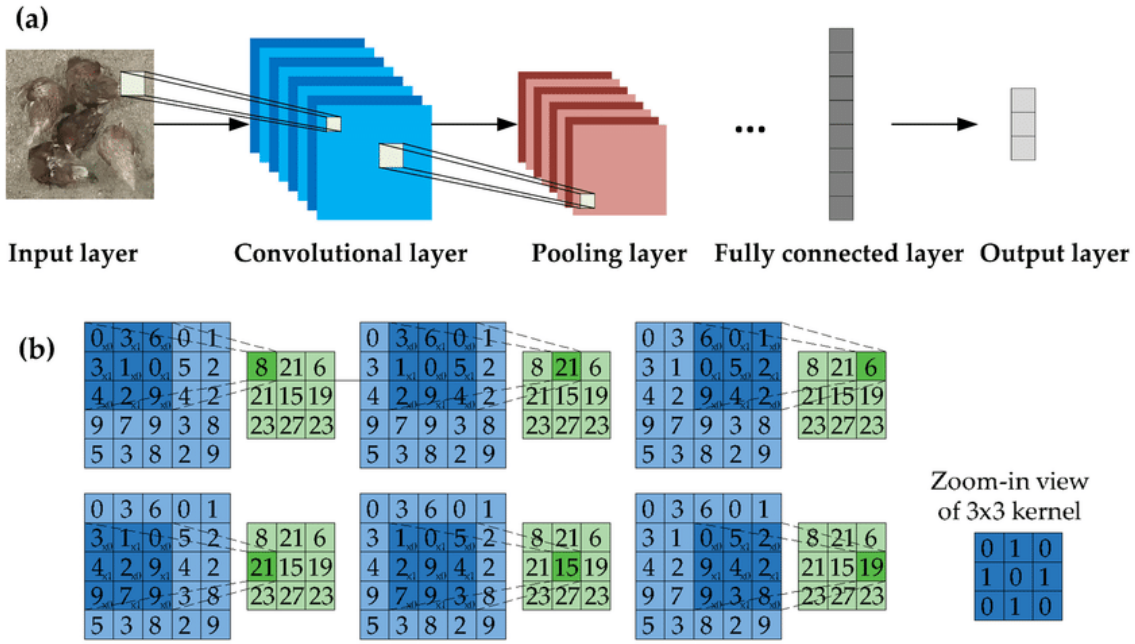
Convolution layers

- This is the first building block of a CNN. The main operation performed here is called convolution, which involves applying a sliding window function to a matrix of pixels representing an image.
- In the convolutional layer, multiple filters of the same size are applied. Each filter is designed to detect specific patterns in the image, such as edges, curves, or the overall shape of the digits.



Convolution layers

- To put it simply, in the convolutional layer, small grids (referred to as filters or kernels) slide over the image, acting like mini magnifying glasses that search for particular patterns, such as lines, curves, or shapes.
- As the grid moves across the image, it creates a new matrix that highlights the locations where these patterns are found.
- For instance, one filter might excel at detecting straight lines, while another might identify curves.
- By combining the results from several filters, the CNN can form a comprehensive understanding of the various patterns within the image.

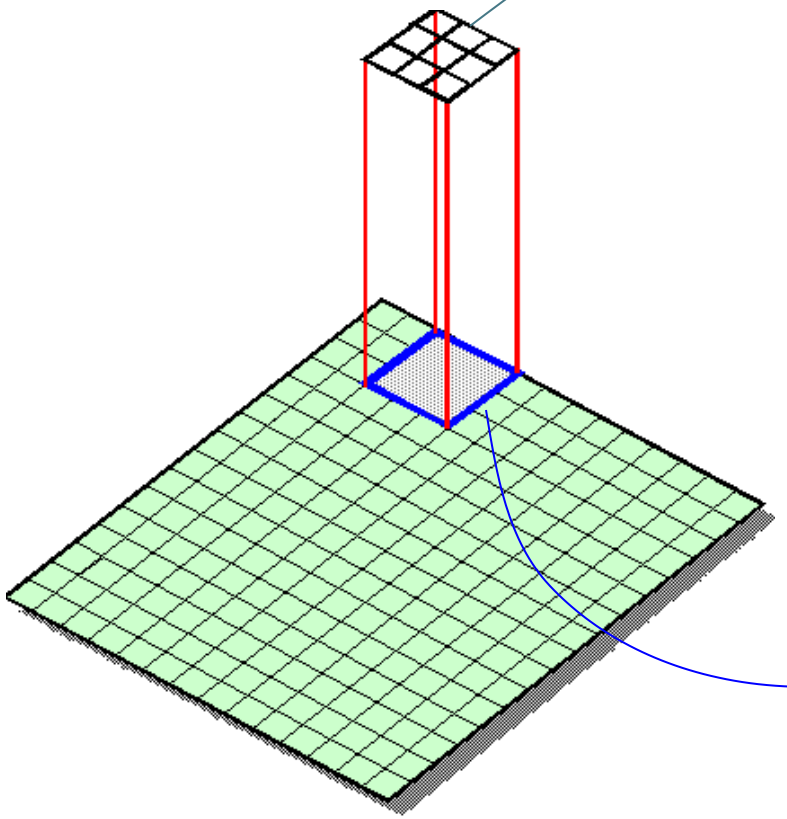




Convolutional Layers

- **Convolution Operation**: The core idea is to use filters (also called kernels) that slide over the input data to produce feature maps. Each filter detects different features like edges, textures, or colors.
- **Stride**: This refers to the step size of the filter as it moves across the input. A stride of 1 means the filter moves one pixel at a time, while a stride of 2 skips every other pixel.
- **Padding**: Sometimes, to preserve the spatial dimensions of the input, padding (adding extra borders of zeroes) is applied around the edges of the input data.

Convolving a mask with an image



3×3 mask

(filter, kernel, template, window)

w_i's are *weights*

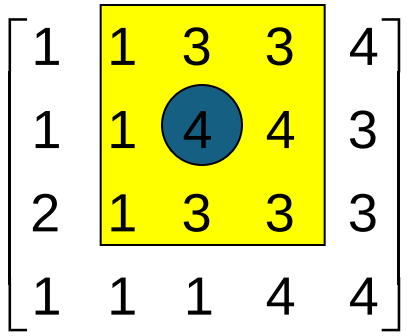
w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

gray levels of pixels under mask

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

Convolution – example

- Given the image and 3×3 mask



0	-1	0
-1	4	-1
0	-1	0

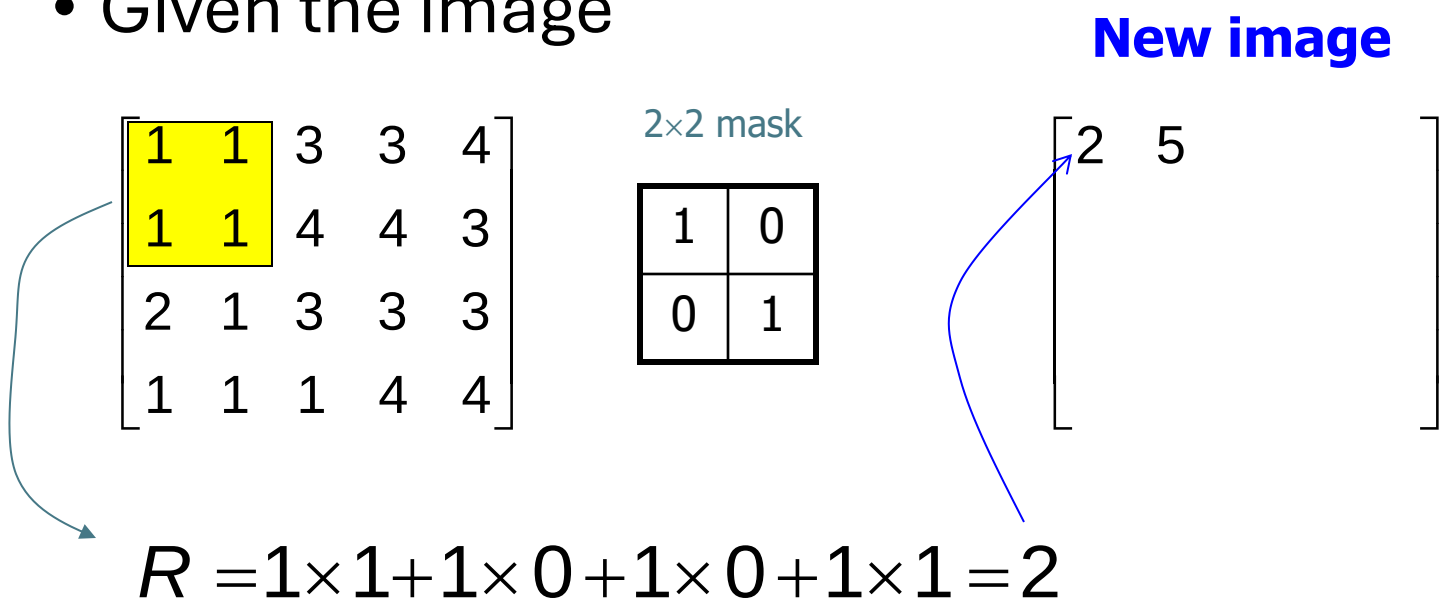
- The response R to the linear mask is

$$R = 1\times 0 + 3\times(-1) + 3\times 0 + 1\times(-1) + 4\times 4 + 4\times(-1) + 1\times 0 + 3\times(-1) + 3\times 0 = 5$$

- So, the response of the neighbourhood of the circled "4", pixel to the mask, is "5"

Convolution – example 2

- Given the image

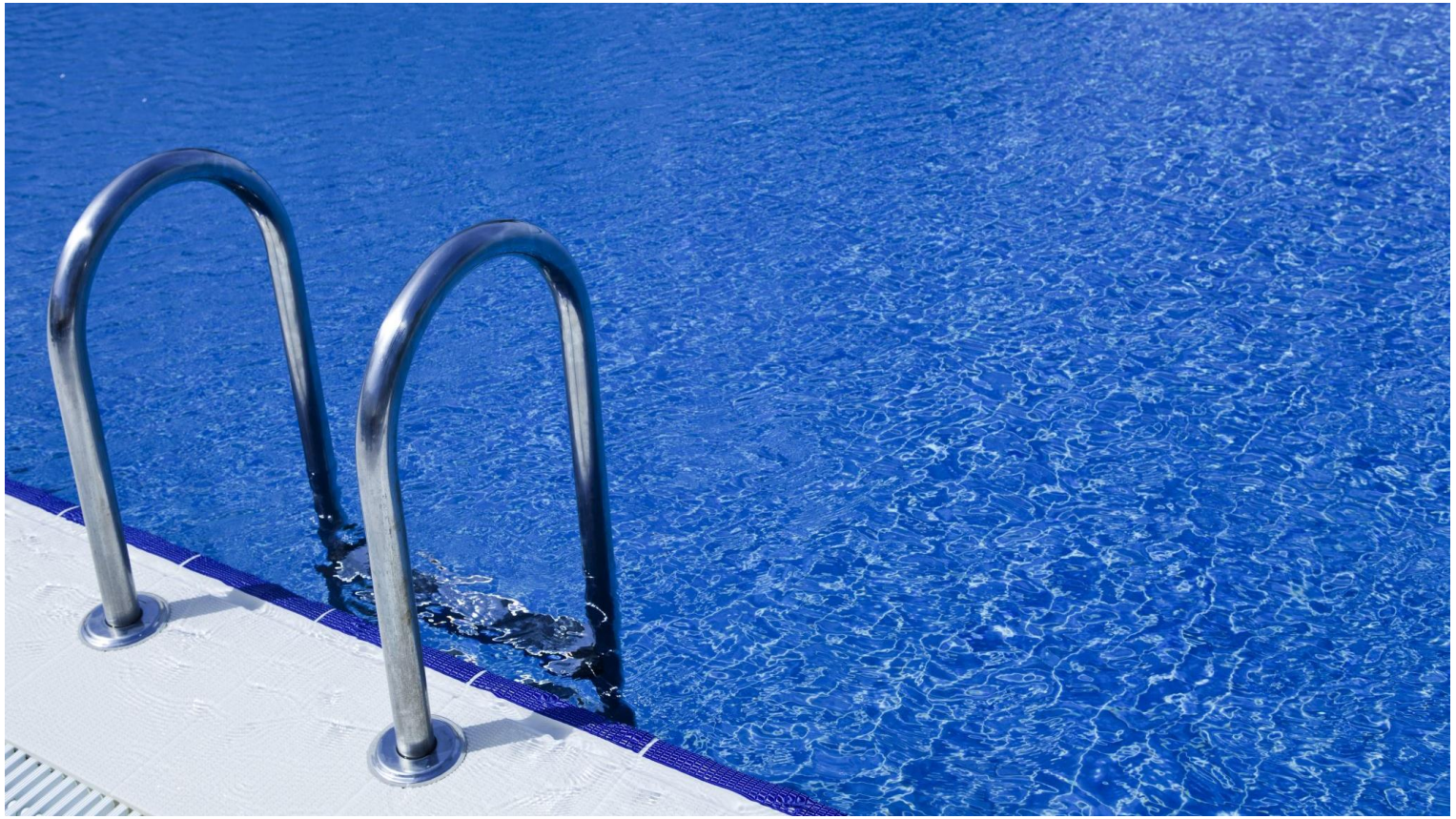


a feature detector

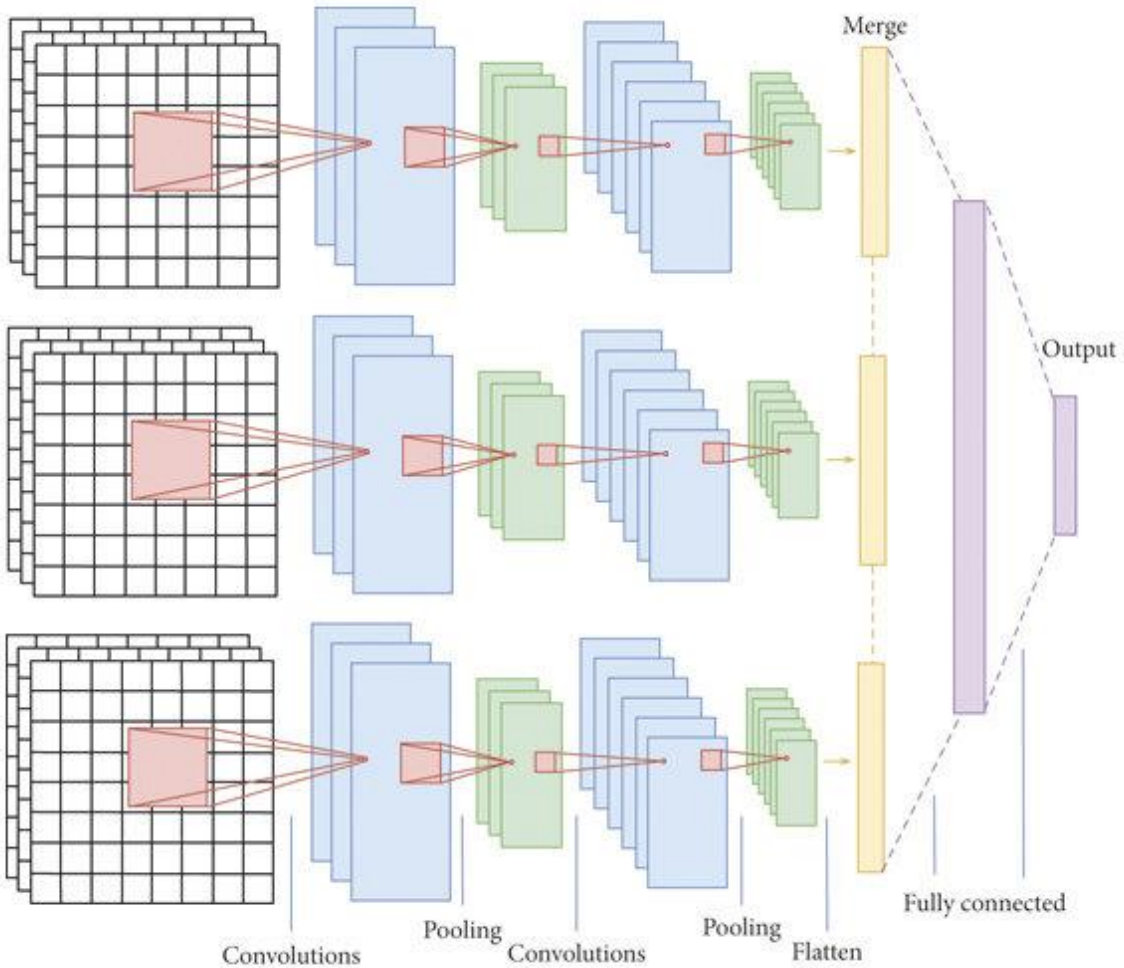
- The kernel is also commonly referred to as a feature detector in the literature because its weights can be adjusted to identify specific features in the input image.
- For example:
  - A kernel that averages neighboring pixels can be used to blur the image.
  - A kernel that subtracts neighboring pixels is used for edge detection.
  - The more convolutional layers a network has, the better it becomes at detecting more complex and abstract features.



2. **Pooling layer**



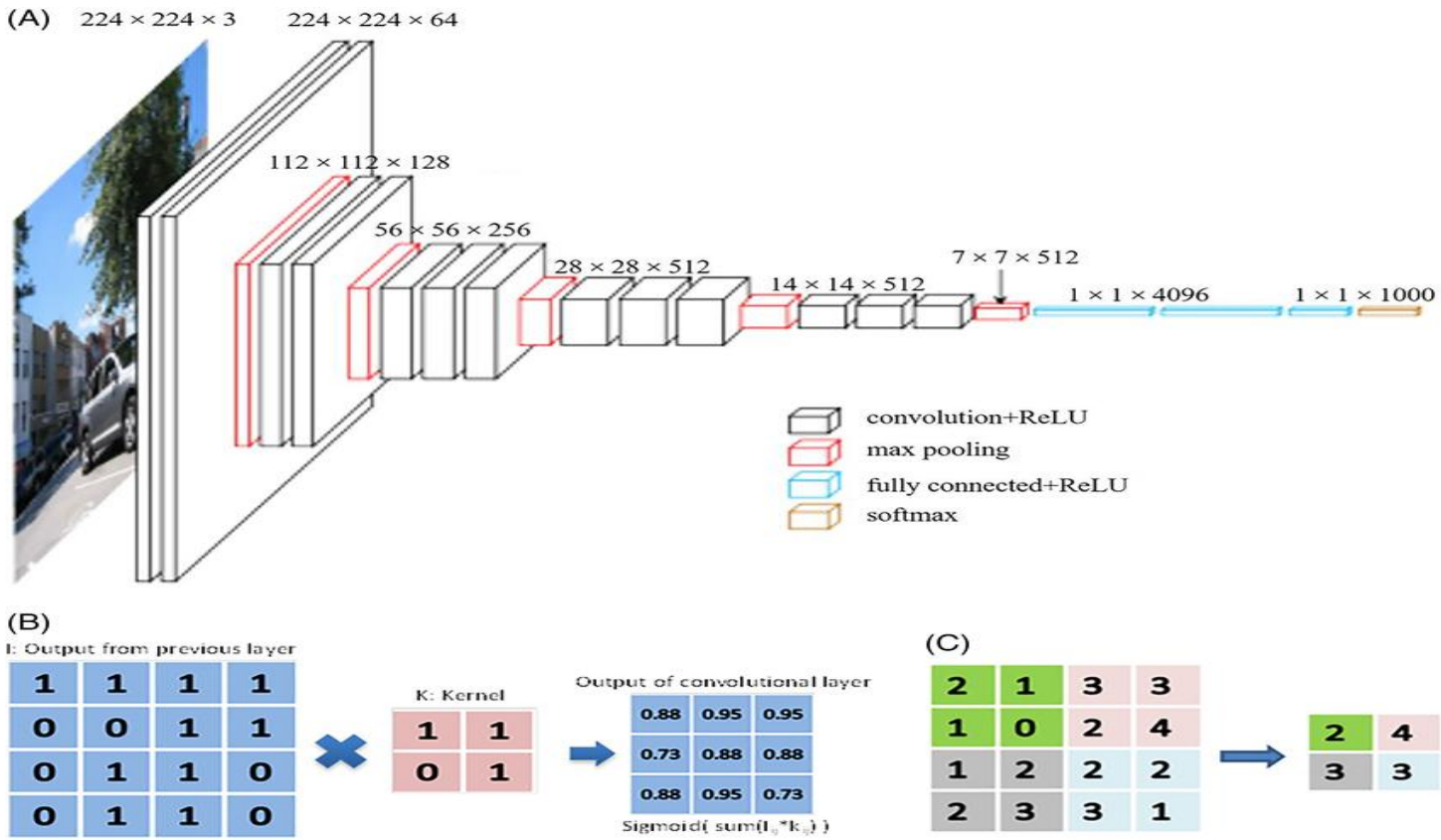
CNN pooling layer



Pooling layer

- The goal of the pooling layer is to pull the most significant features from the convoluted matrix.
- This is done by applying some aggregation operations, which reduce the dimension of the feature map (convoluted matrix), hence reducing the memory used while training the network.
- Pooling reduces the spatial dimensions (width and height) of the feature maps, making the network more manageable and reducing the number of parameters and computations.
- **Pooling is also relevant for mitigating overfitting.**
- The most common aggregation functions that can be applied are:
  - Max pooling: The most common pooling method, where the maximum value within a region of the feature map is taken.
  - Average pooling: Instead of taking the maximum value, the average value of the region is taken.
  - Sum pooling: corresponds to the sum of all the values of the feature map

Max pooling

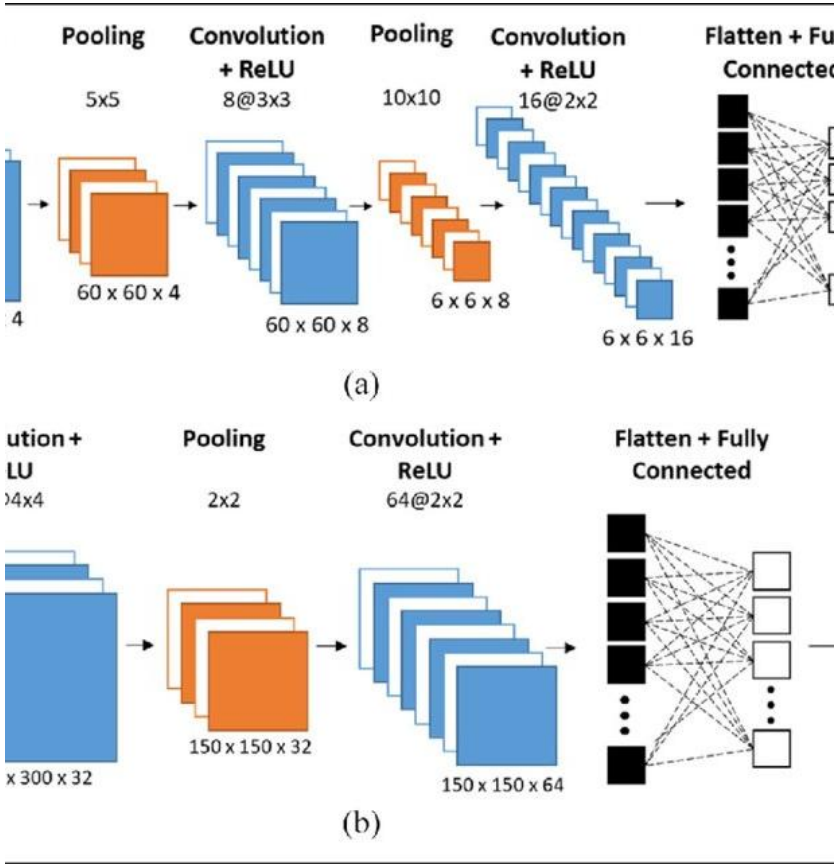

(A)
224 × 224 × 3
224 × 224 × 64
112 × 112 × 128
56 × 56 × 256
28 × 28 × 512
14 × 14 × 512
7 × 7 × 512
1 × 1 × 4096
1 × 1 × 1000
convolution+ReLU
max pooling
fully connected+ReLU
softmax
(B)
I: Output from previous layer
K: Kernel
Output of convolutional layer
Sigmoid(sum(I_ij*k_ij))
(C)

Mitigating overfitting

- Pooling contributes to mitigating overfitting by
  - reducing the complexity of the model,
  - enforcing spatial invariance,
  - making the network more robust to noise, and
  - helping it focus on the most relevant features.
- These factors together make the model more likely to perform well on new, unseen data, rather than memorising the training data.

Mitigating overfitting in CNNs

1. **Reduction of Model Complexity:**
   - Pooling reduces the spatial dimensions (width and height) of the feature maps, which in turn decreases the number of parameters in the subsequent layers of the network. With fewer parameters, the model becomes less complex, which reduces the risk of overfitting to the training data.
2. **Spatial Invariance:**
   - Pooling introduces a degree of spatial invariance, meaning the network becomes less sensitive to small translations or distortions in the input. For example, whether a feature (like an edge) is slightly shifted in the input image, the pooled feature map will still capture that feature effectively. This invariance helps the model generalize better to new, unseen data.
3. **Robustness to Noise:**
   - Max pooling, in particular, helps in reducing the influence of noisy activations. By taking the maximum value in a pooling region, the network focuses on the most prominent features and ignores small variations or noise. This makes the model more robust and less likely to overfit to noise in the training data.

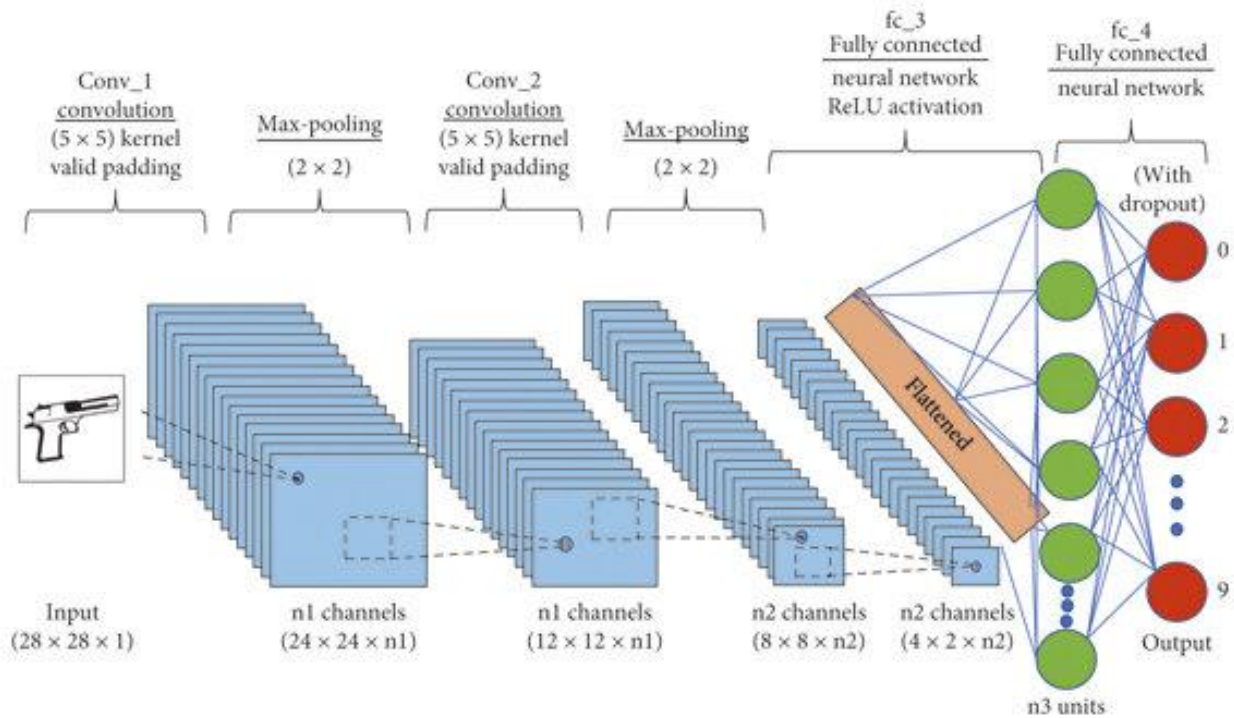


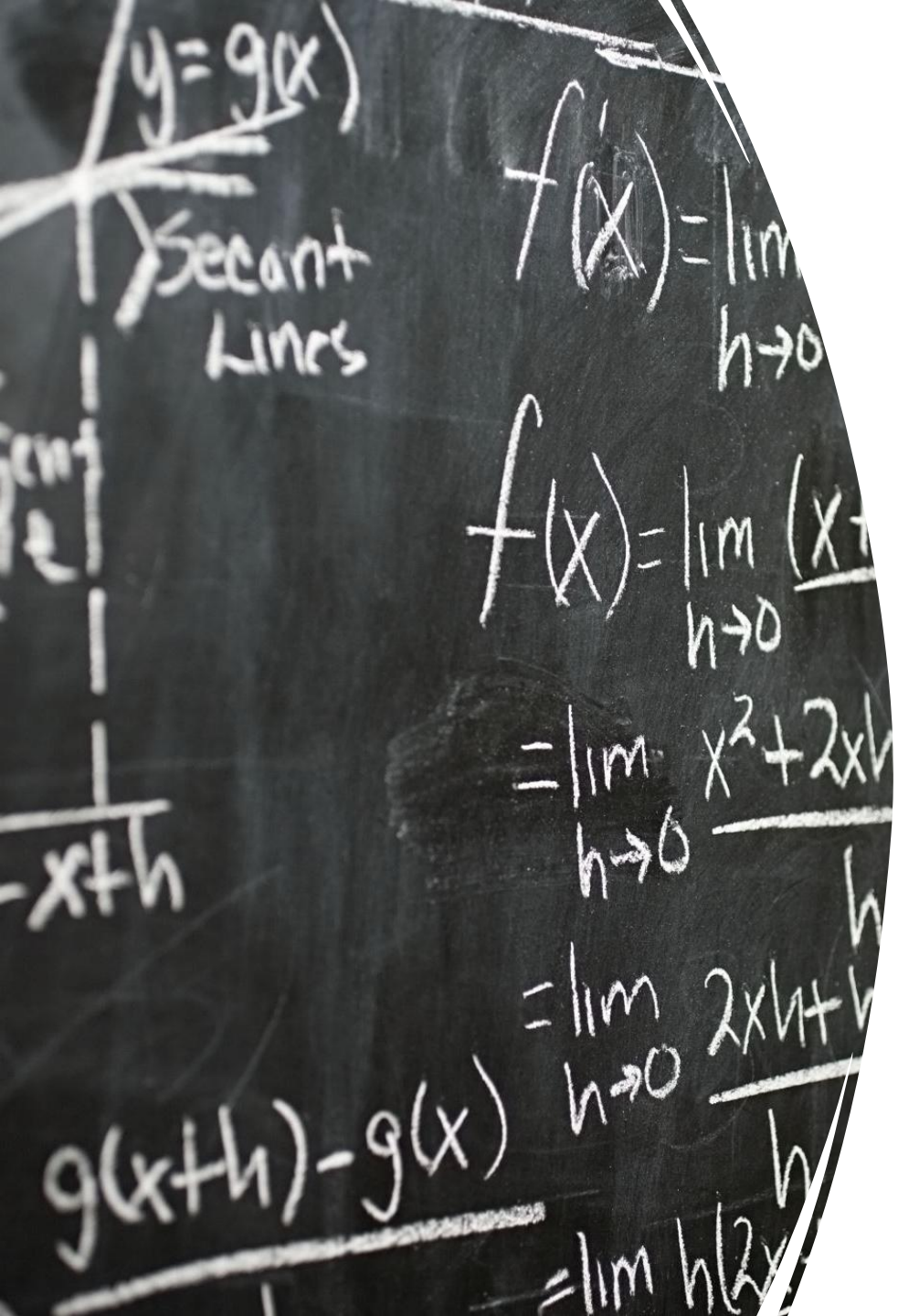
Mitigating overfitting in CNNs

4. **Dimensionality Reduction:**
   - Pooling significantly reduces the size of the feature maps, which reduces the computational load and the amount of information the network needs to process. By distilling the feature maps into more compact representations, the network is forced to focus on the most important features, leading to better generalization.
5. **Implicit Regularization:**
   - While pooling is not a regularization technique in the traditional sense (like dropout or L2 regularization), its effect on reducing the feature map size and enforcing invariance acts as a form of implicit regularization. This **regularization helps prevent the network from fitting too closely to the training data,** which could lead to poor performance on unseen data.
6. **Prevention of Overly Detailed Feature Learning:**
   - Without pooling, the network might learn very fine-grained details that are specific to the training data but not generalizable. Pooling encourages the network to learn more abstract, generalized features, which are less likely to cause overfitting.

Summery: Pooling Layer

- The purpose of the pooling layer is to extract the most important features from the convoluted matrix.
- This is achieved by applying aggregation operations, which reduce the size of the feature map (the convoluted matrix) and, in turn, decrease the memory required during network training.
- Pooling also plays a role in preventing overfitting.
- The most common aggregation methods used in pooling are:
  - **Max pooling:** Selects the maximum value from the feature map.
  - **Sum pooling:** Adds up all the values in the feature map.
  - **Average pooling:** Calculates the average of all the values.
- As the pooling function is applied, the feature map's dimensions become smaller.
- **The final pooling layer flattens the feature map, preparing it for processing by the fully connected layer.**

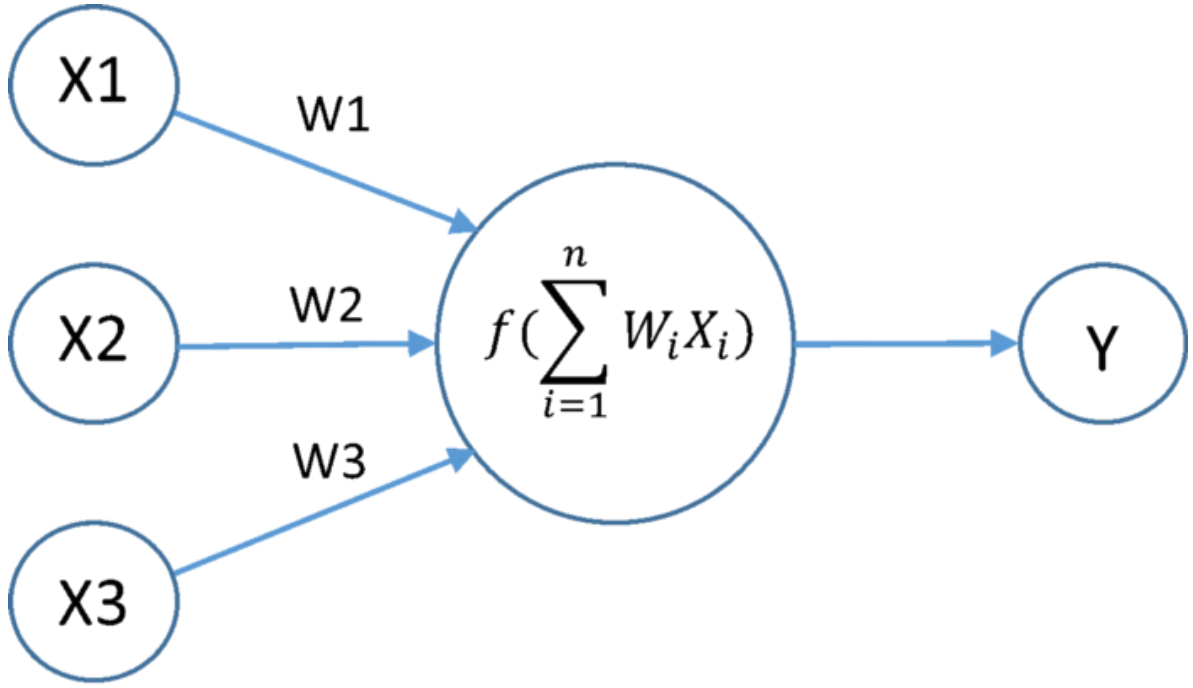

Conv_1
convolution
(5 × 5) kernel
valid padding
Max-pooling
(2 × 2)
Conv_2
convolution
(5 × 5) kernel
valid padding
Max-pooling
(2 × 2)
fc_3
Fully connected
neural network
ReLU activation
fc_4
Fully connected
neural network
(With
dropout)
0
1
2
9
Output
Flattened
Input
(28 × 28 × 1)
n1 channels
(24 × 24 × n1)
n1 channels
(12 × 12 × n1)
n2 channels
(8 × 8 × n2)
n2 channels
(4 × 2 × n2)
n3 units



3. Activation functions

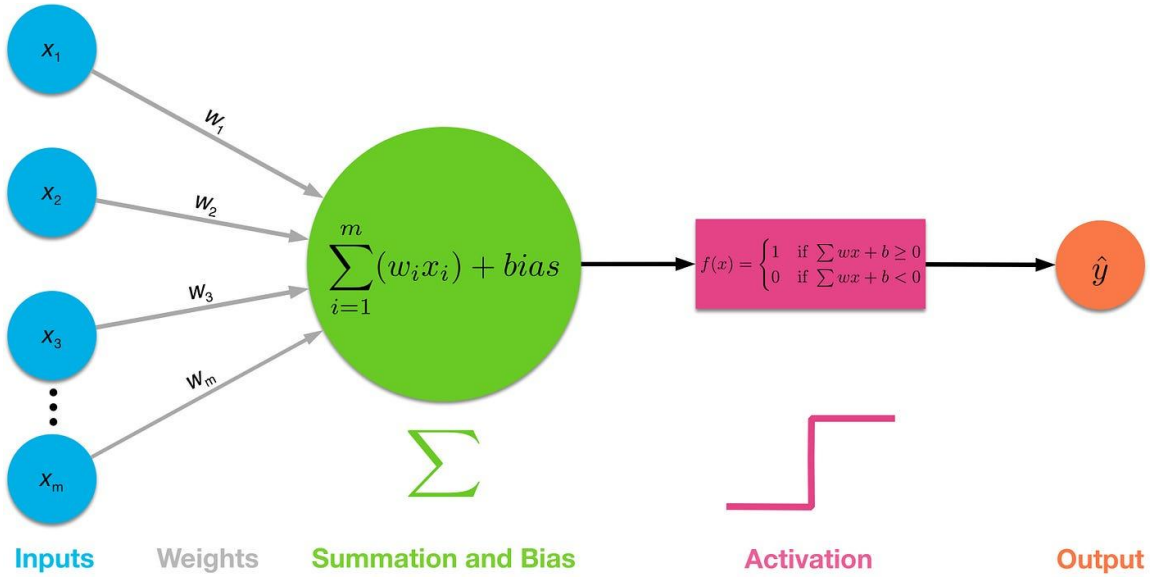
Activation Functions

- **Neurons Produce an Output Based on Their Input**
  Each neuron in a layer receives a weighted sum of inputs (plus a bias). That sum by itself is just a linear combination of the inputs. If we only had linear combinations, no matter how many layers we stacked, our overall network would still effectively be a linear model—unable to learn complex, nonlinear patterns.
- **The Activation Function Adds Non-Linearity**
  After computing the weighted sum, the activation function is applied. Common examples include: ReUL, Sigmoid and Tanh, and there are others as well. The key point is that each of these functions transforms (often in a nonlinear way) the raw sum coming into the neuron.



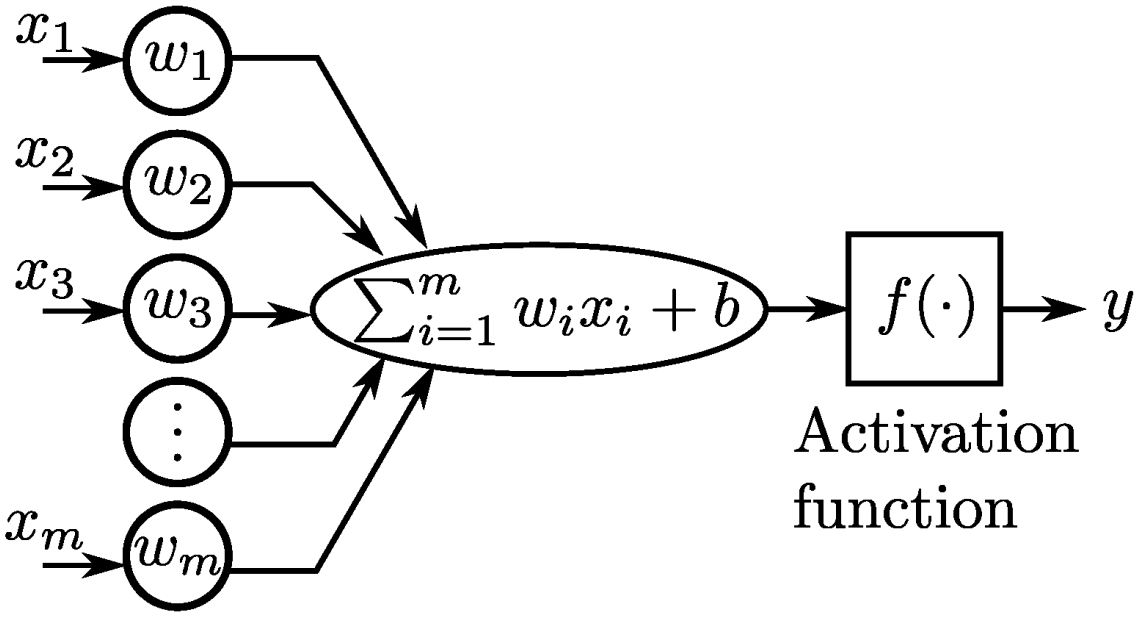
Activation Functions

- **Deciding Which Features Get Passed Forward**
  - Because the activation function can squash or zero out values (in the case of ReLU for negative inputs, or in the case of Sigmoid for very large negative or positive inputs), it effectively decides whether a certain “feature” (i.e., a particular pattern or aspect of the data captured by that neuron’s weights) is strong enough to be carried forward or not.
  - If the transformed output of that neuron is nearly zero, then that feature is effectively “ignored” in subsequent layers.
- **This Matters for Learning**
  - By “turning off” neurons whose output is below some threshold (as happens in ReLU for negative inputs), the network avoids propagating noise or irrelevant features.
  - By saturating outputs for extremely positive or negative inputs (as in Sigmoid or Tanh), the activation function can highlight strong signals.
  - These behaviors help the network focus on the most important patterns and ignore smaller, less important ones.



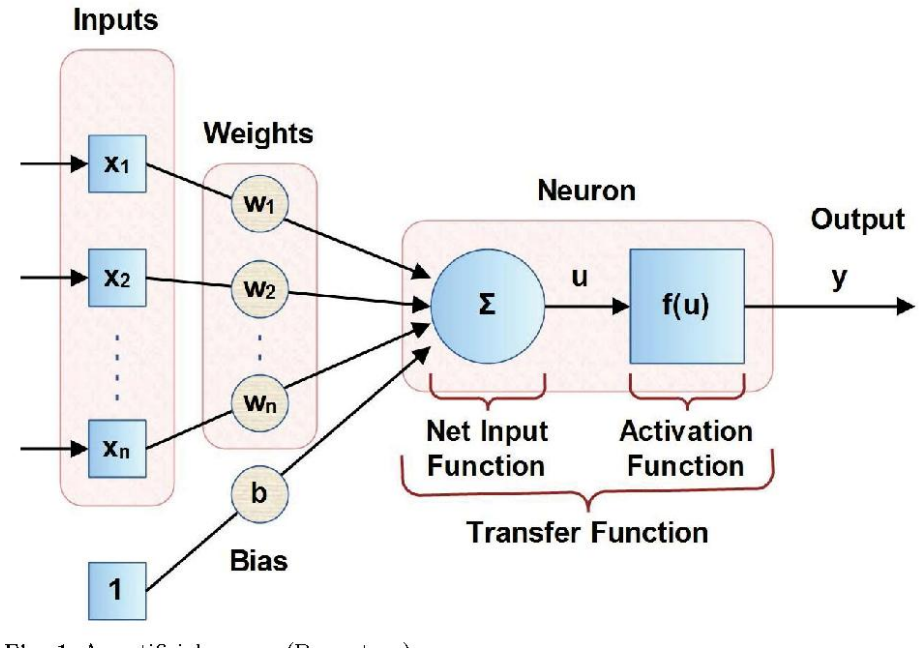
Activation function

- Activation functions play a crucial role in the functioning of neural networks, including Convolutional Neural Networks (CNNs) and other types of deep learning models.
- They determine whether a neuron should be activated or not, essentially deciding which features should be passed forward through the network and which should not.
- This decision-making process allows the network to learn complex patterns and representations from the input data.
- Activation functions are essential for introducing non-linearity, enabling deep learning, shaping outputs, and guiding the learning process in neural networks.
- They are a fundamental component that allows neural networks to learn and model complex data.



Activation function

- The hierarchical feature learning observed in deep neural networks is fundamentally dependent on non-linear activation functions.
- These functions allow the network to build complex, multi-level representations of data, transforming simple input features into high-level abstractions.
- Without non-linearity, the network would be limited to learning only linear relationships, severely restricting its ability to model the complex patterns and interactions that are crucial for tasks like image recognition, natural language processing, and beyond.
- the non-linear activation allows the network to learn a function that correctly maps the inputs to the outputs by combining the features in a non-linear way.



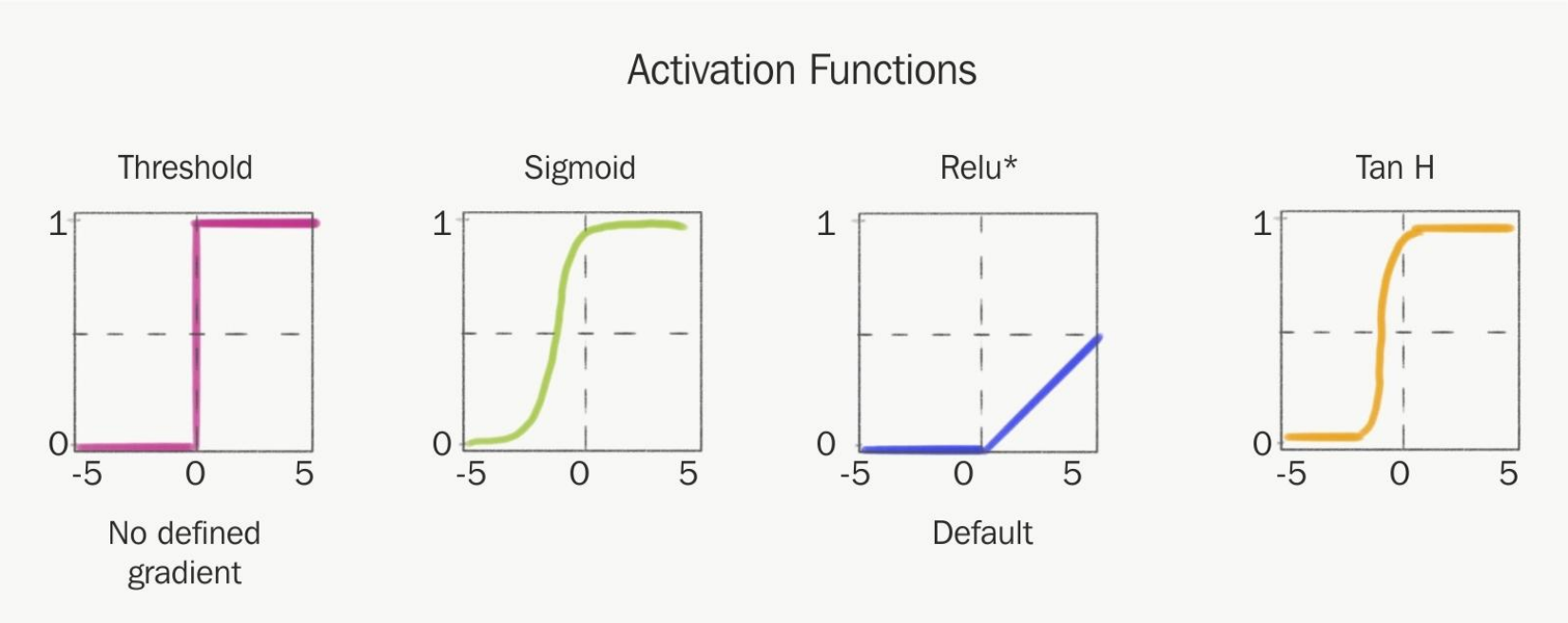
Common Activation Functions

Common Activation Functions:

1. **ReLU (Rectified Linear Unit)**:
   - $f(x) = \max(0, x)$
   - ReLU is widely used because it introduces non-linearity while being computationally efficient. It also helps alleviate the vanishing gradient problem by allowing gradients to propagate through the network more effectively.
2. **Sigmoid**:
   - $f(x) = \frac{1}{1+e^{-x}}$
   - The sigmoid function squashes input values to a range between 0 and 1, making it useful for binary classification problems. However, it can suffer from the vanishing gradient problem, particularly in deep networks.
3. **Tanh (Hyperbolic Tangent)**:
   - $f(x) = \tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
   - Tanh squashes input values to a range between -1 and 1. Like sigmoid, it is used to normalize input, but it centers the data, which can be advantageous. However, it also suffers from the vanishing gradient problem.
4. **Leaky ReLU**:
   - $f(x) = \max(\alpha x, x)$ where α is a small positive constant.
   - Leaky ReLU addresses the "dying ReLU" problem, where neurons can become inactive and stop learning, by allowing a small, non-zero gradient when the input is negative.
5. **Softmax**:
   - $f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}}$
   - Softmax is often used in the output layer of a classification network to convert raw scores into probabilities that sum to one, making it easier to interpret the output as class probabilities.



Common Activation Functions

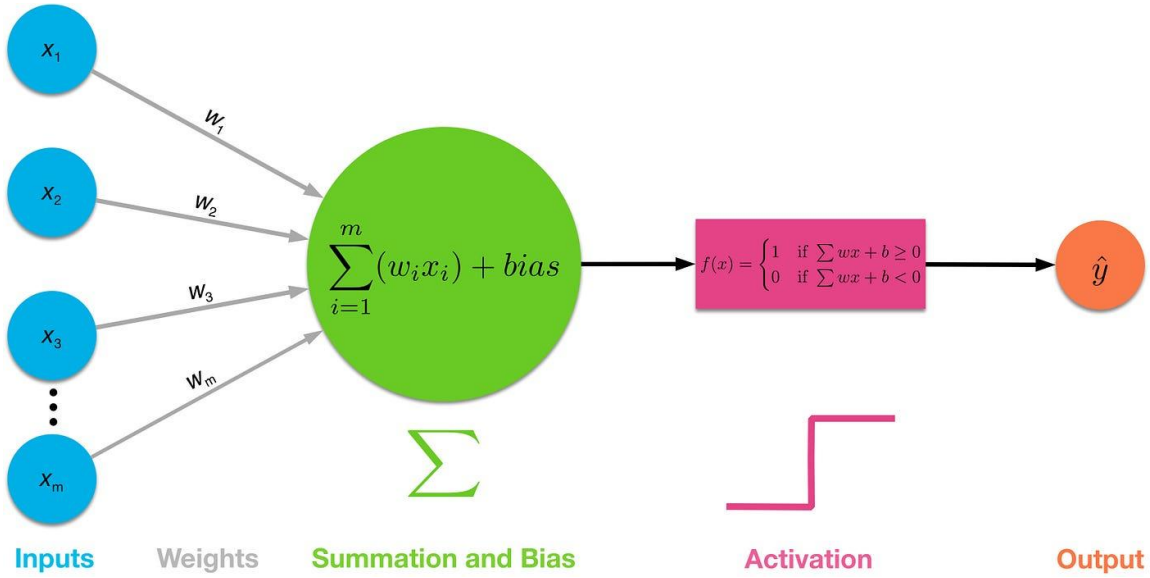


Choosing the Right Activation Function:

- After each convolution operation, a activation function such as ReLU is applied. This function allows the network to capture non-linear relationships between features in the image, enhancing its ability to recognise various patterns. Additionally, it helps prevent the vanishing gradient problem.
- The choice of activation function depends on the specific task, the architecture of the network, and the nature of the data. For instance:
  - **ReLU** and its variants (like Leaky ReLU) are popular for hidden layers in deep networks due to their simplicity and effectiveness.
  - **Sigmoid** or **Tanh** might be used in smaller networks or in the final layer for specific tasks (e.g., binary classification).
  - **Softmax** is almost always used in the output layer for multi-class classification problems.

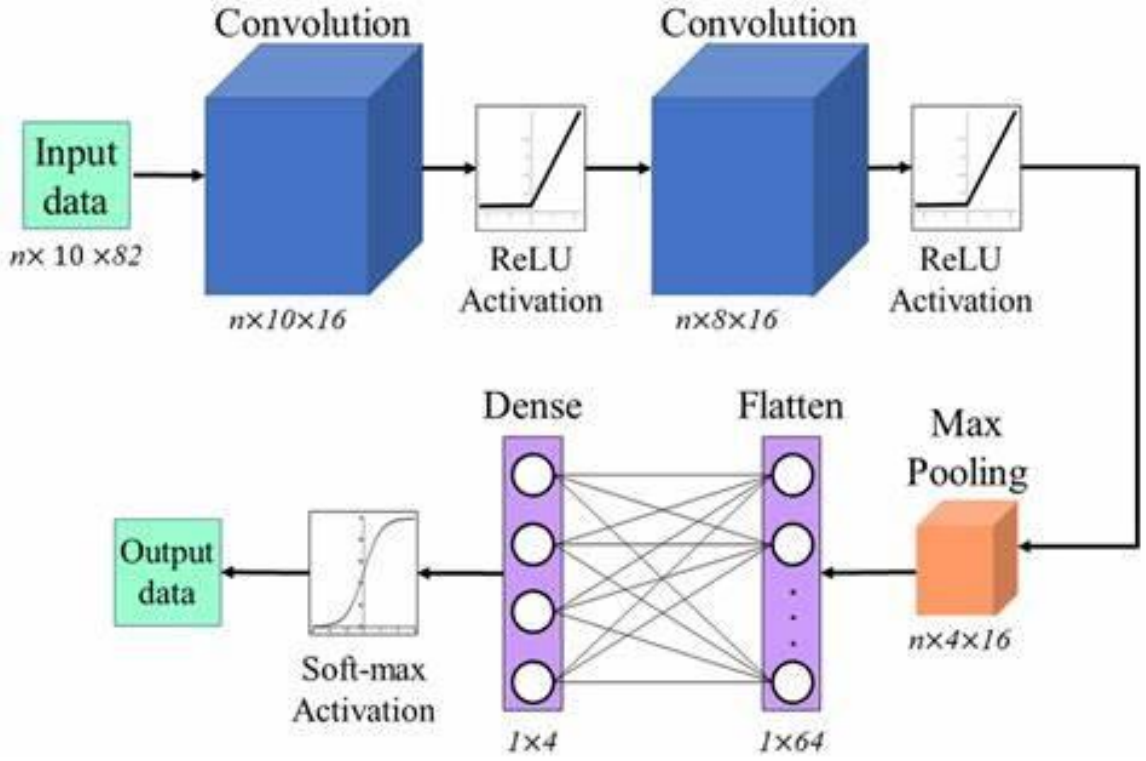
Activation function: ReLU

- **ReLU (Rectified Linear Unit)**: The most common activation function used in CNNs, which replaces negative values with zero.
- Other activation functions like Sigmoid or Tanh can also be used, though ReLU is preferred due to its ability to reduce the likelihood of **vanishing gradients**.
- The **vanishing gradient** problem is a phenomenon that arises during the training of deep neural networks when using backpropagation.
- It occurs because the gradients, calculated by the chain rule, become increasingly small as they are propagated backward through the layers of the network, especially when using activation functions like sigmoid or tanh.
- This problem does not occur in learning algorithms that don't use backpropagation or during the forward pass and inference phases of a neural network.



Activation function: ReLU

- **Stable Gradients**: Unlike sigmoid or tanh, which can have gradients that are much smaller than 1 (and even approach 0), ReLU's gradient of 1 for positive inputs helps maintain strong and stable gradients during backpropagation.
    - This helps mitigate the vanishing gradient problem, especially in deep networks.
- **Efficient Learning**: The gradient of 1 ensures that the weights in the network can be updated efficiently without the risk of very small gradients slowing down the learning process.
    - The gradient of ReLU is 1 for positive inputs because the function is linear in this region (with a slope of 1), and this property is what makes ReLU particularly effective for training deep neural networks.

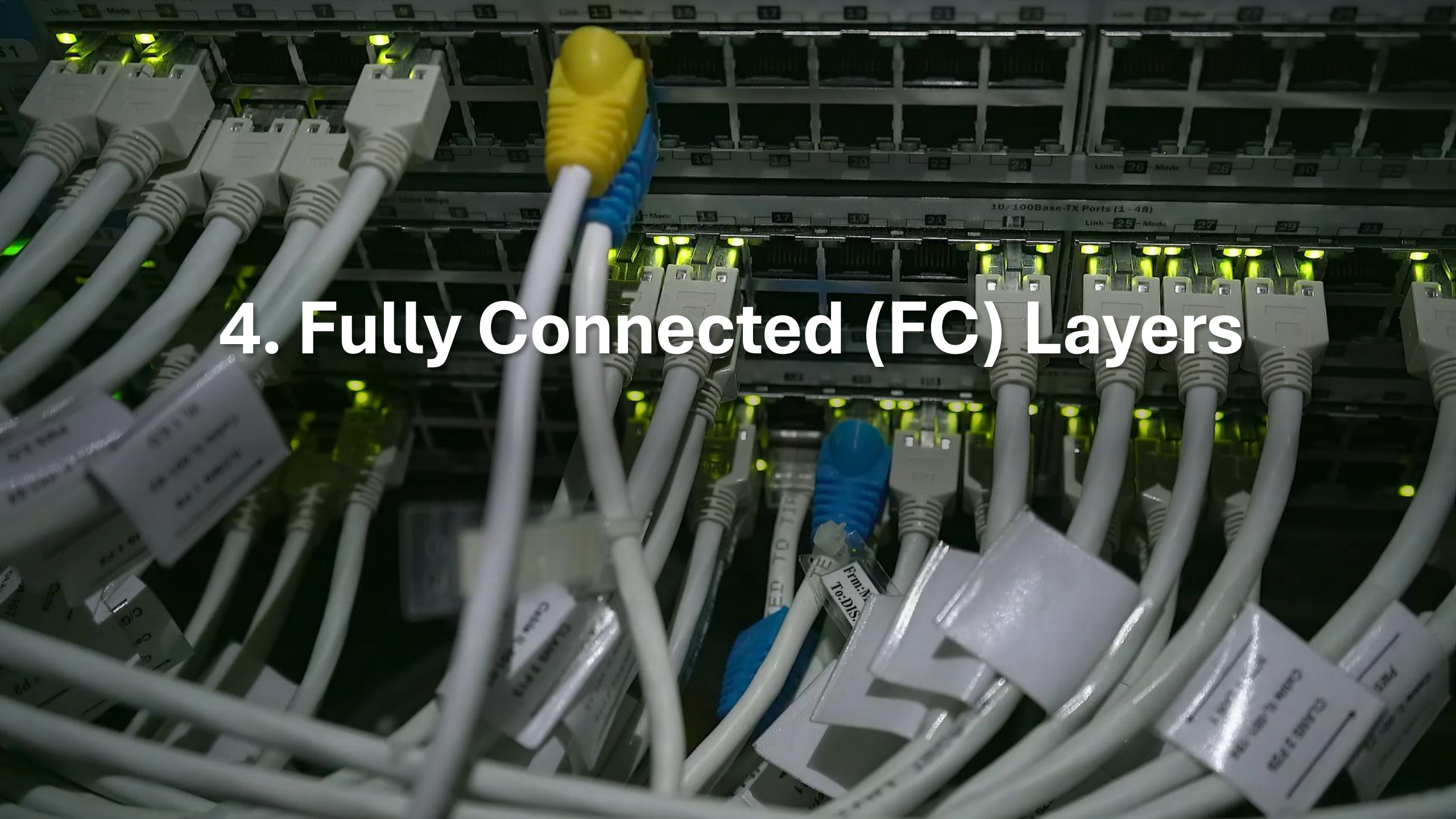


Enabling Hierarchical Feature Learning

- **Example: Image Recognition**
- Consider the task of recognizing an image of a cat:
- **First Layer**: Detects edges and textures in the image.
- **Second Layer**: Combines these edges to detect patterns like fur or specific shapes like the outline of an ear.
- **Third Layer**: Combines these patterns to recognize parts of the cat, like its face, body, or tail.
- **Final Layers**: Integrates all these parts to determine that the image represents a cat.
- At each stage, non-linear transformations enable the network to combine features from the previous layer in increasingly complex ways, moving from simple edges to full object recognition.

Key Roles of Activation Functions

1. **Introducing Non-Linearity**:
    1. Neural networks are composed of layers of neurons, where each neuron computes a weighted sum of its inputs and applies an activation function. Without activation functions, the network would behave like a linear model regardless of its depth, as a series of linear transformations would still result in a linear output.
    2. Non-linearity allows the network to capture complex patterns and interactions in the data, enabling it to model intricate relationships that linear models cannot.
    3. When you introduce a non-linear activation function like ReLU into a neural network, you enable the network to create more complex mappings between inputs and outputs
2. **Enabling Deep Networks**:
    1. Deep networks (those with many layers) rely on activation functions to propagate non-linear transformations across layers. This depth, combined with non-linear activation functions, allows the network to build up increasingly complex representations of the input data, layer by layer.
3. **Controlling the Flow of Information**:
    1. Activation functions control the flow of information in the network by deciding which neurons get activated. This selective activation helps the network focus on important features and ignore irrelevant ones, improving its learning efficiency.
4. **Helping with Gradient-Based Optimization**:
    1. Activation functions influence how gradients are propagated through the network during training. For instance, the choice of activation function affects the gradients computed during backpropagation, impacting how the network learns from the data.
    2. Some activation functions, like ReLU, help mitigate the vanishing gradient problem, which can occur when gradients become too small to drive learning effectively in deep networks.
5. **Shaping the Output**:
    1. In the final layer of a network, activation functions often shape the output into a desired format. For example, a softmax activation function is typically used in the output layer of a classification network to produce probabilities that sum to one, making it easier to interpret the output as class probabilities.



4. Fully Connected (FC) Layers

Fully Connected (FC) Layers:

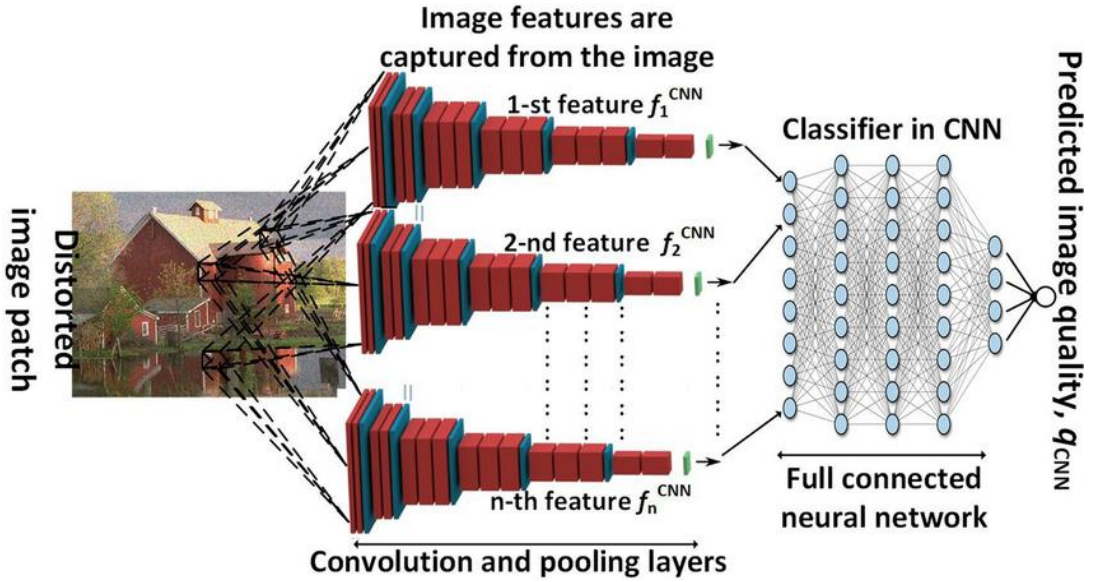
- **Fully Connected (FC) Layers** in Convolutional Neural Networks (CNNs) are dense layers where each neuron (or node) in one layer is connected to every neuron in the next layer.
- Typically, after the convolutional and pooling layers have extracted features from the input (e.g., an image), the network will flatten the resulting feature maps into a 1D vector, and feed that into one or more Fully Connected layers for classification or regression tasks.



2. Structure of Fully Connected Layers

2.1 Dense Connections

- In a dense (or fully connected) layer, each neuron in layer L is connected to *all* the neurons in layer L−1
- This contrasts with convolutional layers, where each neuron is only connected to a *local region* of the previous layer's output.



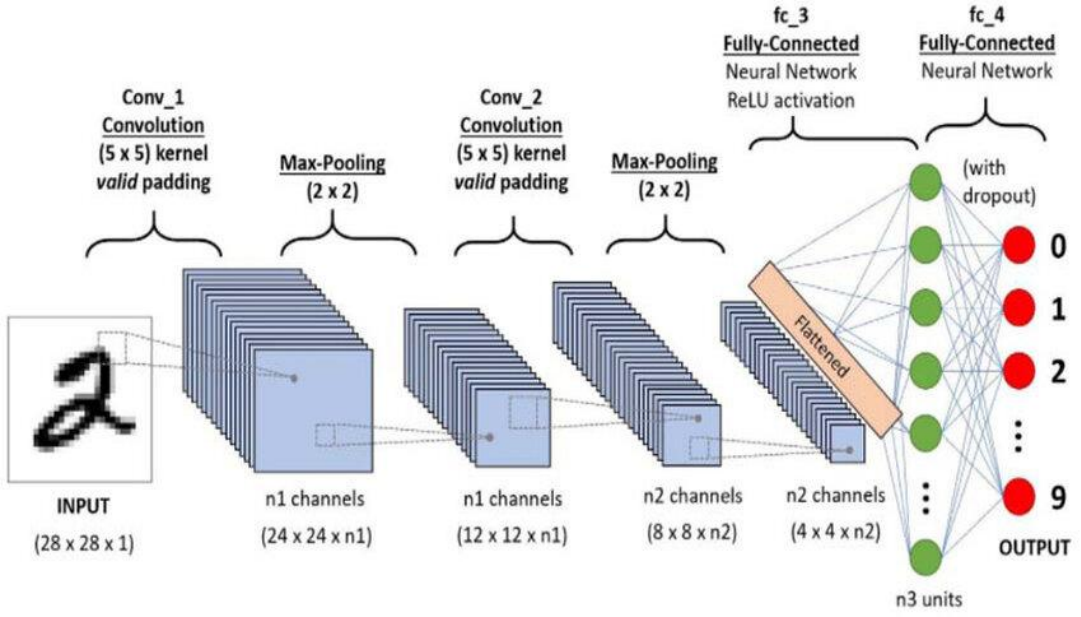
1. Transition from Convolutional Layers to FC Layers

1.1 Feature Extraction (Convolutional + Pooling Layers)

- Early and mid CNN layers apply convolutions and pooling operations to detect hierarchical features in the input image (edges, textures, object parts, etc.).
- The outputs from these layers are typically in a **multi-dimensional** (height × width × channels) form.
- 1.2 Flattering
  - •Before passing these feature maps to a Fully Connected layer, CNNs commonly “flatten” them into a single 1D vector.
  - •The dimension of this vector is the product of the spatial dimensions (height, width) and the number of channels from the final convolutional layer.

1.3 **Feeding Into FC Layers**

The flattened vector is then passed to one or more fully connected (dense) layers.



2. Structure of Fully Connected Layers

2.2. Weights and Biases

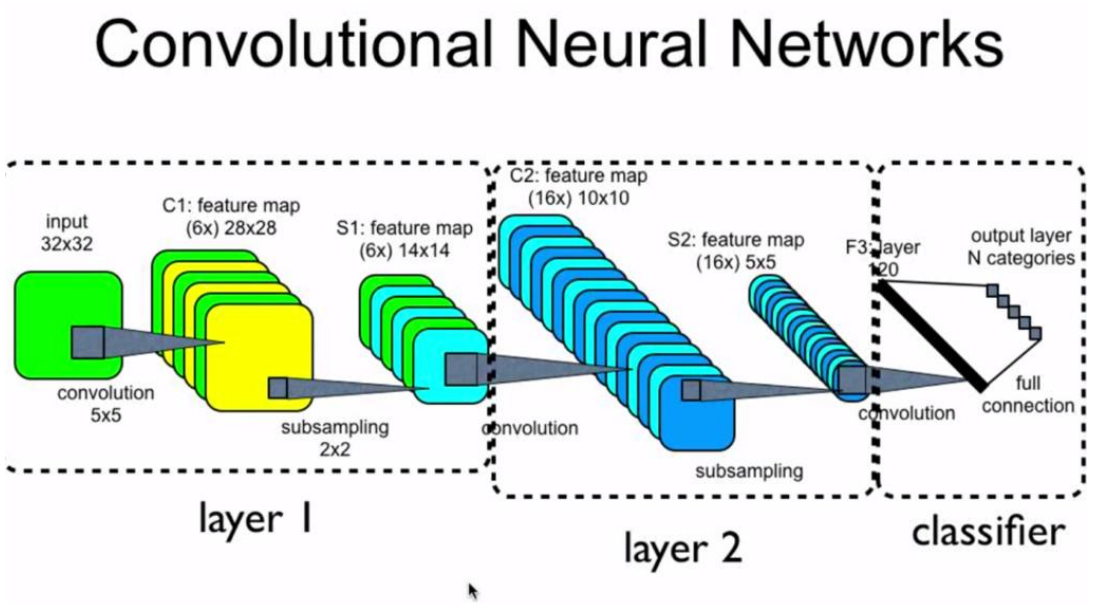
1. Each connection in a fully connected layer has a weight, and each neuron has a bias term.
2. This typically leads to a large number of parameters.
    1. For example, if the flattened vector is size 4096, and the first FC layer has 1000 neurons, that alone accounts for $4096 \times 1000 = 4{,}096{,}000$ weights, plus 1000 biases.

2.3 Activation Function

1. As with convolutional layers, each neuron in an FC layer has an activation function (e.g., ReLU, Sigmoid, Tanh).
2. This allows the layer to learn non-linear decision boundaries.

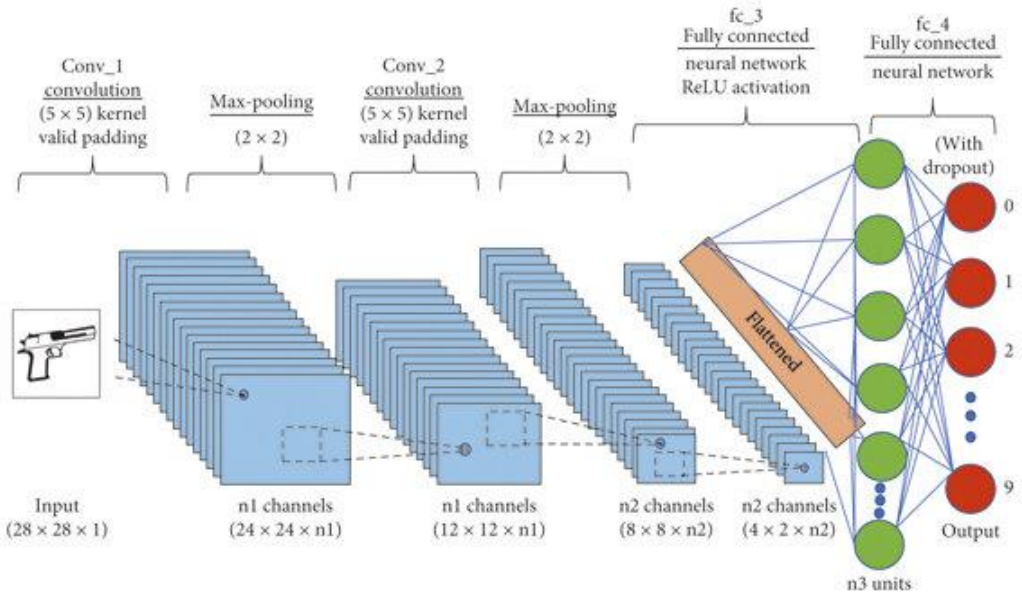
3. Purpose of Fully Connected Layers

- **Combining Features for Classification/Prediction**
  - The Fully Connected layers take the “spatially reduced, feature-rich” representation from the convolutional layers and learn how to map those features to the final output (e.g., class scores in classification tasks).
  - Typically, the final FC layer has as many neurons as the number of classes (if it’s a classification problem).
- **Interpreting Learned Features**
  - While convolutional layers learn lower- and mid-level structures (like edges, textures, object parts), the FC layers combine those structures to form **global** patterns or final category predictions.
- **Decision Making**
  - You can think of the FC layers as the part of the CNN that actually makes the high-level decisions (e.g., “Is this image a cat or a dog?”). The convolutional part is the feature extraction pipeline, while the fully connected part is essentially a high-level classifier that uses those extracted features.



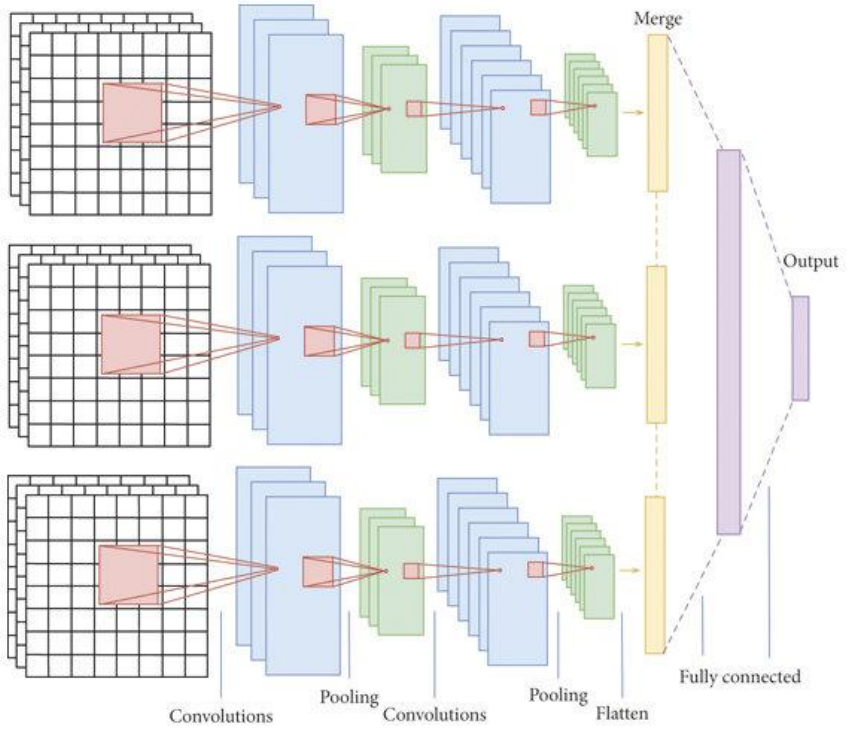
4. Trends and Variations

- **Reduced or Eliminated FC Layers**
  - Many modern architectures (e.g., ResNet, Inception) rely more on global average pooling or other techniques, reducing the need for large, parameter-heavy FC layers.
  - In some cases, a single FC layer might be used just at the end (or none at all if using global average pooling + a final classification layer).
- **Parameter Efficiency**
  - Because FC layers can have a large number of parameters, they can be more prone to overfitting. Methods like **dropout** are often used to regularize these layers.
- **Alternative Architectures**
  - Instead of flattening and using FC layers, some architectures use **fully convolutional** approaches all the way through (e.g., for segmentation tasks).



Fully Connected (FC) Layers:

- These layers are used at the end of the network, where neurons have full connections to all activations in the previous layer. The output from the convolutional and pooling layers is flattened
- The final FC layer usually uses a softmax ac ility distribution over the possible classes (in cla



Fully Connected Layers

- These layers are the final part of the convolutional neural network, receiving input from the flattened one-dimensional matrix produced by the last pooling layer.
- ReLU activation functions are applied here to introduce non-linearity.
- In the end, a softmax layer is used to generate probability values for each possible output label, with the final predicted label being the one with the highest probability.

Summery on FC

- **Fully Connected (FC) Layers** act as the final “classifier” (or regressor) on top of the CNN’s feature extraction pipeline.
- The convolution + pooling layers learn spatial features at various levels of complexity, and the FC layers combine those features to produce the network’s final output.
- While still widely used, many advanced architectures incorporate global average pooling or other strategies to reduce the massive parameter counts that often come with fully connected layers.

5. Dropout



Dropout:

- A regularisation technique used to prevent overfitting by randomly setting a fraction of input units to zero during training

Regularisation

- The term **“regularisation”** comes from mathematics, where it originally referred to methods that “regulate” or **constrain** solutions to make them more stable and less prone to wild fluctuations.
- In machine learning, “regularisation” does something similar:
  1. It **constrains or modifies** how a model learns (e.g., by penalising large weights or dropping neurons).
  2. Those constraints **“regularise”** the model, preventing it from overfitting to noisy details in the training data.
  3. As a result, the model’s learned patterns are more “regular” (simpler and more general), helping it perform better on unseen data.

Regularisation technique

- A **regularisation technique** in machine learning is any method used to **reduce overfitting** and help the model generalize better to new, unseen data.
- **Overfitting** occurs when a model fits the training data too closely—memorising details and noise—resulting in poor performance on data it hasn't seen before.
- Regularization techniques add constraints or modifications to the learning process so the model can't rely on memorising specifics of the training set. Instead, it must learn more **general, robust** patterns.

Examples of regularization include:

  - **L2 and L1 Regularisation** (weight decay): penalising large weights.
  - **Dropout**: randomly "dropping" neurons during training to prevent over-reliance on any single neuron.
  - **Early Stopping**: halting training when validation performance stops improving.

- All of these aim to keep the model simpler or more balanced so it performs better on **unseen** data.

Dropout

- **What is Overfitting?**
  - Overfitting means the network memorizes the training examples too well. It performs great on training data but fails to generalize to new, unseen data.
- **How Does Dropout Work?**
  - During training, dropout randomly “turns off” (sets to zero) a certain percentage of neurons in a layer on each forward pass.
  - For example, if dropout is set to 0.5, about half of the neurons in that layer are randomly ignored each time we feed data through the network.
- **Why Is This Helpful?**
  - By randomly dropping neurons, the network can’t rely too heavily on any single neuron (or small group of neurons) to make predictions.
  - Instead, it has to learn **redundant representations**—where multiple neurons work together to detect useful features.
  - This effectively forces the network to learn more robust patterns that generalize better to new data.
- **When Is Dropout Used?**
  - Often used in fully connected (dense) layers after convolutional layers in CNNs, or in various layers of other architectures.
  - It is most commonly applied during training. During inference (i.e., when making predictions), dropout is usually turned off, and all neurons are used at full capacity.
- **Key Benefit**
  - Dropout helps reduce overfitting by making the neural network “less certain,” encouraging the learning of more general features.

Understanding Dropout

1. **Preventing Overfitting**:
    - Overfitting occurs when a model learns the training data too well, including its noise and specific details, leading to poor generalisation on new, unseen data.
    - Dropout helps mitigate overfitting by introducing randomness into the training process. By randomly dropping out neurons during training, the network cannot rely too heavily on any single neuron or subset of neurons. This forces the network to learn more robust and generalized features.
2. **How Dropout Works**:
    - During each training iteration, a random subset of neurons is set to zero (i.e., they do not contribute to the forward pass and do not participate in backpropagation).
    - The probability of dropping a neuron is controlled by a hyperparameter called the dropout rate (often denoted as ppp). For example, if p=0.5 then each then each neuron has a 50% chance of being dropped during a training iteration.
    - During testing or inference, dropout is not applied. Instead, the outputs of the neurons are scaled by the dropout rate p (or equivalently, the weights are scaled by 1−p) to maintain the expected value of the activations.

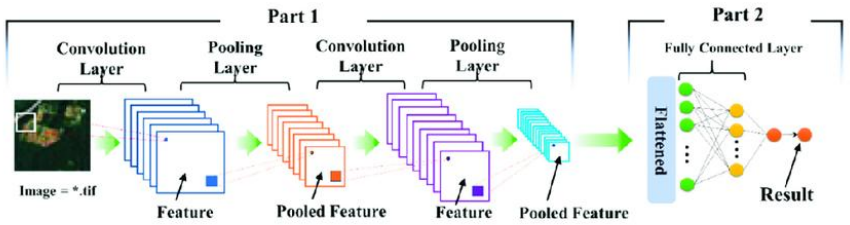
Understanding Dropout

3. Effects on the Network:

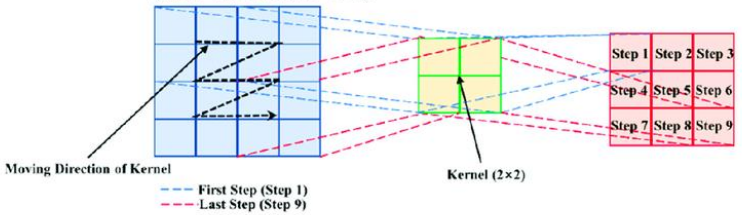
- **Reduces Co-Adaptation**: Dropout prevents neurons from becoming too dependent on specific other neurons, known as co-adaptation. This encourages the network to develop redundant representations and share information across neurons, which leads to better generalisation.
- **Creates an Ensemble Effect**: Dropout can be thought of as training an ensemble of smaller networks that share weights. Each smaller network sees a different subset of the neurons, and during inference, all these networks are averaged, leading to a more robust final model.
- **Increases Robustness**: By randomly disabling parts of the network during training, dropout makes the model more resilient to variations and noise in the data.

Summary

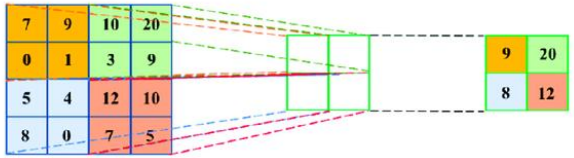
- Dropout is a powerful regularisation technique that helps prevent overfitting by introducing randomness during training. By randomly setting a fraction of input units (neurons) to zero, dropout forces the neural network to learn more generalized features and reduces the model's reliance on specific pathways. This leads to a more robust model that is better able to generalize to new, unseen data.
- In short, **dropout** is like making the network do its work while “dropping” random teammates each round, so it learns to spread the responsibilities across the team. This stops the network from over-relying on a few special neurons and improves overall robustness.



(a) Convolution layer - Feature Extraction from Image
(Stride =1, Kernel size = 2×2)



(b) Pooling Layer - Down Sampling of Feature
(Max Pooling, Stride = 2)



(c) Flatten Layer and Dense Layer

